

FALCON-1 PoC

기술 분석 보고서 (Tech Analysis Report)

영역 D — Manipulation 제어 (7종)

[accent : BURGUNDY]

프로젝트 : FALCON-1 (Flow-Assisted Logistics & Collaborative Operation Network — One)

팀 : NERDLABS | 버전 : v0.1 | 작성일 : 2026-04-30

작성자 : 이건희 (NERDLABS 팀장) | 검토 : CTO 페르소나

보고서 BLUF

BLUF FALCON-1 Manipulation 제어 영역 7종 분석 결과: MoveIt 2 + Pilz Industrial Motion + moveit_task_constructor + ros2_control 4종이 MUST (양팔 OpenArm 제어 표준 stack). MoveIt Servo는 SHOULD (Cartesian 실시간 제어), OMPL은 MoveIt 2에 자동 포함, CHOMP/STOMP는 비권장(OMPL+Pilz로 충분). v1.0 PoC는 Hand-coded grasp + Pilz LIN/CIRC + moveit_task_constructor pick-place 시퀀스로 양팔 동시 동작 (Pattern P2) 구현. RPi5에 Coordinator(MoveIt planning) + L1×2에 ros2_control hardware interface 분산 운용.

본 보고서는 영역 A(미들웨어) + B(PTP 동기) + C(Nav2 mobile)를 전제로 한다. PTP <100μs 동기 위에서 양팔 joint trajectory가 RJ45를 통해 일관 timestamp로 발행되며, Vic Pinky base가 작업 zone으로 이동한 후 OpenArm 양팔이 picking 동작.

본 영역의 accent 색상은 BURGUNDY (정밀 산업 제어 의미)을 사용한다.

영역 D — 7종 요약 매트릭스

#	기술	역할	처리 위치	권장도	적용
D1	MoveIt 2	Motion planning framework	RPi5 Coordinator	★★★★★	MUST
D2	Pilz Industrial Motion	LIN/CIRC 산업 trajectory	RPi5	★★★★★	MUST
D3	moveit_task_constructor	Pick-and-Place task graph	RPi5	★★★★★	MUST
D4	OMPL	Sampling-based planner	MoveIt 2 백엔드	★★★★	MUST (자동)
D5	ros2_control	Hardware abstraction 1kHz	L1-R, L1-L	★★★★★	MUST
D6	MoveIt 2 Servo	Cartesian 실시간 jog	L1 또는 RPi5	★★★★	SHOULD (Sprint 4)

#	기술	역할	처리 위치	권장도	적용
D7	CHOMP / STOMP	최적화 기반 planner	Movelt 백엔드	★★	비권장

D1. Movelt 2

BLUF ★★★★★ MUST 채택. ROS 2 manipulation 표준 framework, OMPL planner + FCL collision detection + IK 솔버 + RViz 시각화 통합. OpenArm 7+1 DoF 양팔에 ros2_control hardware interface 통한 직결 가능. v1.0 hand-coded grasp + Pilz LIN trajectory로 Pick-and-Place 구현, v1.1+ Diffusion Policy/VLA 통합 시에도 Movelt 2가 motion validation layer.

1. 개요

항목	내용
풀네임	Movelt 2
출시	2020년 (ROS 2 Foxy 시작), Jazzy 지원
라이선스	BSD 3-Clause
관리	PickNik Robotics + ROS-Industrial Consortium
구성	Move Group + Planning Scene + IK + Collision + Trajectory
Planner 백엔드	OMPL (default), Pilz, CHOMP, STOMP
IK 솔버	KDL (default), TRAC-IK, BioIK
Collision	FCL (Flexible Collision Library)
공식	https://moveit.picknik.ai/

2. 적용 방법

2.1 설치

```
sudo apt install ros-jazzy-moveit ros-jazzy-moveit-py
sudo apt install ros-jazzy-moveit-resources
sudo apt install ros-jazzy-moveit-task-constructor

# OpenArm MoveIt config (Enactic 공식 또는 자체 생성)
ros2 launch moveit_setup_assistant setup_assistant.launch.py
# → OpenArm URDF 로드 → planning group 생성 → SRDF 저장
```

2.2 OpenArm 양팔 구성 (FALCON-1)

```
# openarm_bimanual.srdf 핵심 발췌
<robot name="openarm_bimanual">
  <!-- Planning Group: 좌팔 -->
  <group name="left_arm">
    <joint name="left_joint_1"/>
    <joint name="left_joint_2"/>
    <!-- ... 7+1 DoF -->
    <chain base_link="base_link" tip_link="left_tcp"/>
  </group>

  <!-- Planning Group: 우팔 -->
  <group name="right_arm">
    <chain base_link="base_link" tip_link="right_tcp"/>
  </group>

  <!-- Planning Group: 양팔 동시 -->
  <group name="both_arms">
    <group name="left_arm"/>
    <group name="right_arm"/>
  </group>

  <!-- 자기 충돌 회피 disable (정상 자세 기반) -->
  <disable_collisions link1="left_link_1" link2="left_link_2" reason="Adjacent"/>
  <!-- ... -->
</robot>
```

2.3 핵심 launch 구성 (Dual-L1 분산)

노드	위치	역할
move_group	RPi5	Movelt 2 메인 노드, IK + planning + collision
planning_scene_monitor	RPi5	환경 모델 (octomap, AprilTag obstacles)
servo_node (D6)	RPi5 또는 L1	Cartesian jog (선택)
controller_manager (left)	L1-Left	ros2_control left arm hardware

노드	위치	역할
controller_manager (right)	L1-Right	ros2_control right arm hardware
joint_state_broadcaster	L1-L, L1-R	양팔 joint state 발행
joint_trajectory_controller	L1-L, L1-R	Trajectory 실행

2.4 양팔 동시 pick-and-place 호출 (Python)

```
from moveit.planning import MoveItPy

moveit = MoveItPy(node_name="falcon1_moveit_py")
left_arm = moveit.get_planning_component("left_arm")
right_arm = moveit.get_planning_component("right_arm")
both = moveit.get_planning_component("both_arms")

# 양팔 동시 pose target
left_arm.set_goal_state(pose_stamped_msg=left_grasp_pose, pose_link="left_tcp")
right_arm.set_goal_state(pose_stamped_msg=right_grasp_pose, pose_link="right_tcp")

# 양팔 동시 plan (충돌 회피 통합)
plan_result = both.plan()
if plan_result:
    moveit.execute(plan_result.trajectory, controllers=[])
```

3. 하드웨어 필요·제약

항목	최소	권장	FALCON-1 적합성
CPU (move_group)	duo+ 1.5 GHz	quad+ 2 GHz	❑ RPi5에서 marginal — L1 이전 검토
RAM	1 GB	2-4 GB	✔ RPi5 8GB
IK 계산	1-10 ms	—	✔ 7 DoF는 KDL 충분
Planning 시간	0.5-3 s	—	✔ 양팔 OMPL ~1 s
GPU	불필요 (core)	—	✔
URDF + SRDF	필수	—	✔ Enactic 공식

항목	최소	권장	FALCON-1 적합성
ros2_control	필수	—	✔D5 통합

3.1 Coordinator 위치 결단 (RPI5 vs L1)

옵션	장점	단점	권장
RPi5 Coordinator	대칭, safety + nav과 통합	Movelt 부하 50%+ 시 marginal	Sprint 3 측정 후 결정
L1-Right Coordinator	GPU + CPU 여유	비대칭 (L1-R 부하 증가)	RPi5 부하 초과 시 fallback

4. 장점

- 1. ROS 2 manipulation 사실상 단일 표준
- 2. OpenArm 공식 통합 — Enactic 이 ros2_control hardware_interface + Movelt config 제공 가정
- 3. Multi-arm planning native — both_arms group 으로 양팔 동시 trajectory 충돌 회피
- 4. Plug-in 자유 — Planner (OMPL, Pilz, CHOMP, STOMP) 동적 교체
- 5. RViz 시각화 — 디버깅·발표 영상 모두 활용
- 6. Python + C++ API — 빠른 prototyping + 실시간 제어 양립
- 7. Industry adoption — Universal Robots, Franka, Kinova, Doosan 등 모두 사용

5. 단점·한계

- 8. 학습곡선 매우 가파름 — URDF/SRDF + Movelt config + planning scene + controller manager 동시 이해
- 9. Setup Assistant GUI 까다로움 — 양팔 SRDF 자동 생성 시 self-collision 검증 부담

- 10. Planning 시간 변동 — OMPL 은 sampling 기반이라 같은 query 에 1-3 초 변동 (Pilz LIN 으로 보완)
- 11. RPi5 부하 — move_group + planning_scene_monitor 동시 50%+, Nav2 와 경합
- 12. Trajectory smoothing 한계 — Pilz 또는 후처리 smoother 필요
- 13. Collision world 동적 업데이트 — octomap 또는 AprilTag obstacle 갱신 latency 100-500ms
- 14. Real-time 제어 분리 — MoveIt 은 planning, 1kHz 실시간은 ros2_control 별도

6. 대안 비교

항목	MoveIt 2	ROS 1 MoveIt	Drake (Toyota)	PyBullet planning
ROS 2 native	✔	✗	별도	별도
OpenArm 통합	✔Enactic 공식	✗EOL	✗	✗
Multi-arm	✔	✔	✔	수동
산업 채택	표준	EOL	Toyota·일부	PoC
FALCON-1	✔MUST	비권장	비권장 (학술)	비권장

7. FALCON-1 적용 결정

결정 항목	값
채택 여부	MUST 채택
도입 시점	Sprint 2 (HW Setup, OpenArm 통합 직후)
Coordinator 위치	1차 RPi5, 부하 초과 시 L1-Right로 이전
Planning Group	left_arm, right_arm, both_arms
기본 Planner	OMPL RRTConnect (자유 grasp), Pilz LIN (직선 transport)

결정 항목	값
IK 솔버	KDL (1차) → TRAC-IK (Sprint 4 검토)
Collision	FCL + AprilTag obstacle dynamic
주요 의존성	D2 Pilz, D3 moveit_task_constructor, D5 ros2_control, A1 ROS 2
Sprint Backlog 영향	Sprint 2-3에 ~15 stories

8. 참고

- 공식 문서 : <https://moveit.picknik.ai/>
- Tutorials : <https://moveit.picknik.ai/main/doc/tutorials/tutorials.html>
- OpenArm Moveit config : https://github.com/enactic/openarm_moveit_config (가정)
- PickNik 블로그·발표 : ROSCon 2023, 2024
- Setup Assistant :
https://moveit.picknik.ai/main/doc/examples/setup_assistant/setup_assistant_tutorial.htm
|

D2. Pilz Industrial Motion Planner

BLUF ★★★★★ MUST 채택. 산업용 LIN(직선) · CIRC(원호) · PTP(joint) trajectory 생성, ISO 10218 준수 deterministic motion 보장. OMPL sampling 기반의 변동성을 보완하여 같은 query 에 같은 결과 보장 (reproducibility). v1.0 Pick-and-Place의 transport 단계 (Box 위로 이동) · place 단계 (descent · release)에 핵심.

1. 개요

항목	내용
풀네임	Pilz Industrial Motion Planner
관리	Pilz GmbH (독일 산업 안전 기업) + ROS-Industrial
라이선스	Apache 2.0
MovelIt 통합	MovelIt 2 plugin (planner adapter)
motion 종류	PTP (joint), LIN (Cartesian 직선), CIRC (원호)
결정성	deterministic — 같은 입력 → 같은 출력
표준 준수	ISO 10218-1/2 산업 로봇 motion

2. 적용 방법

2.1 설치

```
# MoveIt 2와 함께 자동 설치
sudo apt install ros-jazzy-pilz-industrial-motion-planner

# planning_pipeline 설정에 추가
# config/moveit_planning.yaml
default_planning_pipeline: ompl
planning_pipelines:
  - ompl
  - pilz_industrial_motion_planner  # ★ 추가
```

2.2 LIN trajectory 사용 (Python)

```
from moveit_msgs.msg import MotionPlanRequest, Constraints, JointConstraint
from geometry_msgs.msg import PoseStamped

# Pilz LIN motion 요청
request = MotionPlanRequest()
request.pipeline_id = "pilz_industrial_motion_planner"
request.planner_id = "LIN"  # PTP / LIN / CIRC
request.group_name = "right_arm"
request.max_velocity_scaling_factor = 0.3
request.max_acceleration_scaling_factor = 0.3

# Cartesian 목표 pose
goal_pose = PoseStamped()
goal_pose.header.frame_id = "base_link"
```

```
goal_pose.pose.position.x = 0.5
goal_pose.pose.position.y = 0.2
goal_pose.pose.position.z = 0.4
goal_pose.pose.orientation.w = 1.0

# 호출
right_arm.set_goal_state(pose_stamped_msg=goal_pose, pose_link="right_tcp")
plan = right_arm.plan(planning_pipeline="pilz_industrial_motion_planner")
```

2.3 FALCON-1 사용 시나리오

단계	Motion	이유
Approach (Pre-Grasp)	PTP (joint)	빠른 이동, OpenArm 효율
Grasp (Cartesian descent)	LIN	직선 정확, 충돌 회피 보장
Lift	LIN	수직 직선 lift, 물체 미끄러짐 방지
Transport	PTP 또는 LIN	거리 길면 PTP, 정밀 영역은 LIN
Place descent	LIN	Box 위 정밀 하강
Retreat	LIN	수직 후퇴

3. 하드웨어 필요·제약

항목	요구	FALCON-1 적합성
MovelIt 2	필수 (D1 의존)	✔MUST
URDF/SRDF	정확한 joint limit + velocity 정의	✔Enactic 공식
계산 시간	<10 ms (deterministic)	✔OMPL 대비 100x 빠름
RAM	수 MB 추가	✔
GPU	불필요	✔

4. 장점

- 15. Deterministic — 같은 입력 → 같은 출력, 재현 가능 (OMPL 은 sampling 변동)
- 16. 산업 표준 motion — LIN/CIRC/PTP 는 KUKA · ABB · Fanuc 모두 사용하는 산업 motion type
- 17. 계산 빠름 — <10ms 로 OMPL 의 100x
- 18. ISO 10218 정합 — 산업 안전 인증 지원 narrative
- 19. OpenArm 같은 collaborative arm 에 적합 — 사람 옆 deterministic motion 이 안전
- 20. Trajectory smoothing 자동 — velocity/acceleration limit 준수
- 21. Pick-and-Place 의 transport·place 는 LIN 이 정답 — 곡선 motion 부적합

5. 단점·한계

- 22. Free-space planning 제한 — 장애물 회피는 OMPL 이 강함, Pilz 는 직선·원호만
- 23. Singularity 회피 미흡 — 7 DoF redundancy 활용 약함
- 24. CIRC motion 사용 빈도 낮음 — 인테리어 마감 작업에 원호 motion 거의 없음
- 25. Multi-waypoint 부족 — 복잡 trajectory 는 OMPL 후 smoothing 이 더 자연스러움
- 26. Hand-tuning 의무 — velocity/acceleration scaling 작업별 수동 조정

6. 대안 비교

항목	Pilz	OMPL	Movelt Servo (D6)
방식	Industrial deterministic	Sampling-based	Cartesian jog
Motion type	LIN/CIRC/PTP	free-space	Cartesian velocity
계산 시간	<10 ms	0.5-3 s	<1 ms
재현성	✔deterministic	✗sampling	—
장애물 회피	약함	✔	—

항목	Pilz	OMPL	MoveIt Servo (D6)
FALCON-1 사용	transport/place	approach 자유	선택적 jog

7. FALCON-1 적용 결정

결정 항목	값
채택 여부	MUST 채택
도입 시점	Sprint 3 (Single Mission v0)
사용 단계	Grasp descent · Lift · Place descent · Retreat (LIN)
Velocity scaling	0.3 (시연 안전), 0.5 (Sprint 5 burn-in)
Acceleration scaling	0.3
OMPL과의 분담	OMPL = approach (자유 free-space), Pilz = 정밀 직선 motion
주요 의존성	D1 MoveIt 2 (필수)

8. 참고

- 공식 wiki :
https://moveit.picknik.ai/main/doc/examples/pilz_industrial_motion_planner/pilz_industrial_motion_planner.html
- Pilz GmbH : <https://www.pilz.com/>
- ROS-Industrial Pilz : https://github.com/PilzDE/pilz_industrial_motion
- ISO 10218 표준 : <https://www.iso.org/standard/73934.html>

D3. moveit_task_constructor (MTC)

BLUF ★★★★★ MUST 채택. Pick-and-Place 같은 multi-stage manipulation task를 task graph 형태로 분해하여 단계별 planning. v1.0 Scene A1 양팔 동시 pick-and-place 핵심 도구. 각 stage(approach → grasp → lift → transport → place)가 별도 planner 사용 가능 (approach 는 OMPL, transport는 Pilz LIN 등).

1. 개요

항목	내용
풀네임	MoveIt Task Constructor
라이선스	BSD
관리	Robert Haschke (Bielefeld Univ.) + PickNik
설계 철학	Hierarchical task decomposition + multi-planner combination
입력	task graph (stages + alternatives + parallels)
출력	validated trajectory sequence
MoveIt 통합	moveit_msgs::Task action server

2. 적용 방법

2.1 설치

```
sudo apt install ros-jazzy-moveit-task-constructor-core
sudo apt install ros-jazzy-moveit-task-constructor-msgs

# Demo 패키지로 학습
ros2 launch moveit_task_constructor_demo demo.launch.py
```

2.2 FALCON-1 Pick-and-Place task graph (C++)

```
// FALCON-1 right arm pick task
Task task;
task.stages()->setName("pick driver");

// Stage 1: Current state
auto current = std::make_unique<stages::CurrentState>("current");
task.add(std::move(current));
```

```

// Stage 2: Open gripper
auto open = std::make_unique<stages::MoveTo>("open gripper", joint_planner);
open->setGroup("right_gripper");
open->setGoal("open");
task.add(std::move(open));

// Stage 3: Approach object (OMPL free-space)
auto approach = std::make_unique<stages::Connect>("approach", ompl_planner);
approach->setTimeout(5.0);
task.add(std::move(approach));

// Stage 4: Generate grasp pose (from AprilTag)
auto grasp_gen = std::make_unique<stages::GenerateGraspPose>("grasp pose");
grasp_gen->setObject("driver");
grasp_gen->setAngleDelta(M_PI / 12);
grasp_gen->setMonitoredStage(task.stages()->findChild("current"));
task.add(std::move(grasp_gen));

// Stage 5: Pick container – Approach + Close + Lift (Pilz LIN)
auto pick = std::make_unique<stages::Pick>("pick", std::move(grasp_gen));
pick->setApproach(0.10, geometry_msgs::Vector3(0,0,-1)); // 10cm down LIN
pick->setLift(0.05, geometry_msgs::Vector3(0,0,1)); // 5cm up LIN
pick->properties().set("object", "driver");
task.add(std::move(pick));

// Plan + Execute
if (task.plan(5)) {
    task.introspection().publishSolution(*task.solutions().front());
    task.execute(*task.solutions().front());
}

```

2.3 양팔 동시 task (Pattern P2)

```

Task dual_task;

// Parallel container – 양팔 동시 실행
auto parallel = std::make_unique<containers::Alternatives>("dual_pick");

// Right arm pick driver
auto right_pick = create_pick_subtask("driver", "right_arm");
parallel->add(std::move(right_pick));

// Left arm pick hammer (동시)
auto left_pick = create_pick_subtask("hammer", "left_arm");
parallel->add(std::move(left_pick));

dual_task.add(std::move(parallel));
dual_task.plan(10); // 충돌 회피 통합 plan

```

3. 하드웨어 필요·제약

항목	요구	FALCON-1 적합성
MovelIt 2	필수	✓D1
CPU	duo+ 1.5 GHz	✓RPi5 또는 L1
RAM	200 MB+	✓
Planning 시간	전체 task 1-5 s	✓시연 시간 영향 미미
디버깅	RViz introspection	✓task graph 시각화

4. 장점

- 27. Multi-stage decomposition — 복잡 task 를 stage 단위로 디버깅·재시도
- 28. Multi-planner combination — approach=OMPL, lift=Pilz LIN 등 stage 별 최적 planner
- 29. Alternatives container — 한 stage 실패 시 대체안 자동 시도
- 30. Parallel container — 양팔 동시 task 통합 plan
- 31. RViz introspection — 각 stage 결과 시각화로 디버깅 직관
- 32. Reusable subtasks — pick subtask 정의 후 양팔에 재사용
- 33. Failure recovery — 실패 stage 식별 후 부분 재계획

5. 단점·한계

- 34. 학습곡선 가파름 — task graph 개념 + C++ API 복잡
- 35. Python API 약함 — 주로 C++ 사용, Python wrapper 미성숙
- 36. Documentation 빈약 — example 부족
- 37. Solver 선택 의존 — stage 별 planner mismatch 시 trajectory 불연속
- 38. 디버깅 까다로움 — 어느 stage 에서 실패했는지 추적 필요
- 39. PickNik 의존성 — maintainer 변동 시 영향 큼

6. 대안 비교

항목	MTC	Behavior Tree (Nav2 BT)	수동 sequence (Python)
Task graph	✔핵심 기능	✔	수동
Movelt 통합	✔native	✗	✔
Multi-planner	✔	✗	수동
Manipulation 특화	✔	Nav 특화	—
FALCON-1	✔MUST	Nav2에서 사용	비권장

7. FALCON-1 적용 결정

결정 항목	값
채택 여부	MUST 채택
도입 시점	Sprint 3 (Single Mission v0)
사용 시나리오	Scene A1 양팔 동시 Pick-and-Place
Stage 구성	current → open → approach (OMPL) → grasp → lift (Pilz LIN) → transport → place (Pilz LIN) → retreat
양팔 통합	Parallel container로 좌·우 동시 plan
Failure recovery	Alternatives container로 retry max 2회
주요 의존성	D1 Movelt 2, D2 Pilz, D4 OMPL

8. 참고

- GitHub : https://github.com/moveit/moveit_task_constructor
- ROSCon 2018 발표 : <https://vimeo.com/293539179>

- Tutorials :
https://moveit.picknik.ai/main/doc/examples/moveit_task_constructor/moveit_task_constructor_tutorial.html

D4. OMPL — Open Motion Planning Library

BLUF ★★★★★ MUST (자동, MoveIt 2 default 백엔드). Sampling-based motion planner의 사실상 표준, RRTConnect · RRT* · BIT* 등 다수 알고리즘 포함. MoveIt 2가 기본 활용하므로 별도 작업 거의 없음. v1.0 approach 단계 (free-space)에 사용, transport-place는 Pilz LIN으로 분담.

1. 개요

항목	내용
풀네임	Open Motion Planning Library
라이선스	BSD
관리	Rice University Kavraki Lab
알고리즘	RRT, RRTConnect, RRT*, BIT*, FMT*, KPIECE 등 30+
MoveIt 2 통합	ompl_interface (default planner)
언어	C++

2. 핵심 알고리즘 비교 (FALCON-1 권장)

알고리즘	특성	FALCON-1 권장
RRT	기본, 빠름, sub-optimal	비권장
RRTConnect	양방향 tree, 빠름	✓ default 권장
RRT*	asymptotic optimal, 느림	Sprint 4 검토

알고리즘	특성	FALCON-1 권장
BIT*	Batch informed, 정확	Sprint 5 검토
KPIECE	kinodynamic 우수	v1.1+

3. 설정 (FALCON-1 권장)

```
# config/ompl_planning.yaml
planner_configs:
  RRTConnectkConfigDefault:
    type: geometric::RRTConnect
    range: 0.0          # 0 = auto

  RRTstarkConfigDefault:
    type: geometric::RRTstar
    range: 0.0
    goal_bias: 0.05

right_arm:
  default_planner_config: RRTConnectkConfigDefault
  planner_configs:
    - RRTConnectkConfigDefault
    - RRTstarkConfigDefault

left_arm:
  default_planner_config: RRTConnectkConfigDefault

both_arms:
  default_planner_config: RRTConnectkConfigDefault
```

4. 장점·단점 요약

장점 :

- MoveIt 2 default — 별도 설치 작업 없음
- 30+ 알고리즘 동적 교체 가능
- 학술적 검증 — 1000+ 논문에서 baseline
- Free-space planning 에 강함 (장애물 회피)

단점 :

- Sampling 기반 — 같은 query 에 다른 결과 (Pilz 로 보완)

- Trajectory 부드럽지 않음 — smoother 필요
- Tuning 까다로움 — range, goal_bias 등 파라미터
- Singularity 회피 약함 — TRAC-IK 등 IK 솔버 보완

5. FALCON-1 적용 결정

결정 항목	값
채택 여부	MUST (MoveIt 2 자동 포함)
도입 시점	Sprint 2 (MoveIt 2 설치와 동시)
기본 알고리즘	RRTConnect
사용 단계	Approach (free-space), 자유 grasp pose
Pilz와의 분담	OMPL = free-space, Pilz = LIN/CIRC
주요 의존성	D1 MoveIt 2

6. 참고

- 공식 : <https://ompl.kavrakilab.org/>
- MoveIt 2 OMPL 가이드 :
https://moveit.picknik.ai/main/doc/examples/ompl_interface/ompl_interface_tutorial.html
- Sucan, Moll, Kavraki 2012 논문 (IEEE RAM)

D5. ros2_control

BLUF ★★★★★ MUST 채택. ROS 2 hardware abstraction 표준, 1kHz 실시간 joint 제어 + controller 동적 로딩. OpenArm 7+1 DoF × 2팔에 ros2_control hardware_interface (Enactic 공식 가정) 통한 직결, joint_trajectory_controller로 MoveIt trajectory 실행. Dual-L1 분산 운용에

서 L1-Right · L1-Left 각각 controller_manager 운용.

1. 개요

항목	내용
풀네임	ros2_control
라이선스	Apache 2.0
관리	ros2_control maintainers (Stogl Robotics 등)
설계	controller_manager + hardware_interface + plugin controller
주기	표준 100Hz, 권장 1kHz (PREEMPT_RT)
Plug-in controllers	joint_trajectory · joint_state_broadcaster · forward_command · effort 등 20+

2. 적용 방법

2.1 설치

```
sudo apt install ros-jazzy-ros2-control ros-jazzy-ros2-controllers
sudo apt install ros-jazzy-controller-manager
sudo apt install ros-jazzy-joint-trajectory-controller
sudo apt install ros-jazzy-joint-state-broadcaster
```

2.2 OpenArm hardware_interface 핵심 (FALCON-1)

```
// openarm_hardware/src/openarm_system.cpp 발췌
namespace openarm_hardware {

class OpenArmSystem : public hardware_interface::SystemInterface {
public:
    CallbackReturn on_init(const HardwareInfo& info) override;
    std::vector<StateInterface> export_state_interfaces() override;
    std::vector<CommandInterface> export_command_interfaces() override;

    return_type read(const Time& time, const Duration& period) override {
        // CAN-FD에서 joint state 수신 (1kHz)
        can_->receive(joint_states_);
        return return_type::OK;
    }
}
```

```

return_type write(const Time& time, const Duration& period) override {
    // CAN-FD로 joint command 전송
    can_->send(joint_commands_);
    return return_type::OK;
}

private:
    std::shared_ptr<CanFdDriver> can_;
    std::vector<double> joint_states_;
    std::vector<double> joint_commands_;
};

} // namespace

```

2.3 controller_manager YAML (Dual-L1)

```

# L1-Right: ~/openarm_right_controllers.yaml
controller_manager:
  ros__parameters:
    update_rate: 1000          # 1 kHz
    use_sim_time: false

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    right_arm_controller:
      type: joint_trajectory_controller/JointTrajectoryController

    right_gripper_controller:
      type: position_controllers/GripperActionController

right_arm_controller:
  ros__parameters:
    joints:
      - right_joint_1
      - right_joint_2
      - right_joint_3
      - right_joint_4
      - right_joint_5
      - right_joint_6
      - right_joint_7
    command_interfaces: [position]
    state_interfaces: [position, velocity, effort]
    state_publish_rate: 100.0
    action_monitor_rate: 50.0
    constraints:
      goal_time: 1.0
      stopped_velocity_tolerance: 0.01

```

3. 하드웨어 필요·제약

항목	최소	권장	FALCON-1 적합성
CPU	duo+ 1.5 GHz	quad+ 2 GHz	✓ L1-R, L1-L
RAM	100 MB	500 MB	✓
커널	표준 OK (100 Hz)	PREEMPT_RT (1 kHz)	□ PREEMPT_RT 빌드 필요 (Sprint 2)
Hardware	USB-CAN, EtherCAT, etc.	—	✓ OpenArm USB-CAN-FD
URDF + ros2_control 태그	필수	—	✓ Enactic 공식
Update rate	100 Hz min	1000 Hz	□ PREEMPT_RT로 1 kHz 검증

4. 장점

- 40. ROS 2 표준 — manipulation hardware abstraction 의 사실상 단일 표준
- 41. Plug-in controller 동적 교체 — joint_trajectory ↔ forward_command ↔ effort 변경 자유
- 42. Multi-machine 자연 분산 — L1-R · L1-L 각자 controller_manager 운용
- 43. MoveIt 2 통합 native — joint_trajectory_controller 가 MoveIt action server
- 44. 실시간 1 kHz — PREEMPT_RT 커널에서 deterministic
- 45. Hardware lifecycle — on_init / on_activate / on_deactivate 명확
- 46. OpenArm 공식 hardware_interface (Enactic) — 자체 작성 부담 없음

5. 단점·한계

- 47. PREEMPT_RT 의존 — 표준 커널에서 jitter 1-5ms
- 48. 학습곡선 — controller_manager + hardware_interface + URDF + YAML 동시 이해
- 49. 디버깅 어려움 — controller 실패 시 원인 파악 어려움

- 50. Plug-in 호환성 — 일부 controller 가 ROS 2 Jazzy 에 미포팅 가능
- 51. 실시간 thread 안전 — read/write 함수에서 lock 사용 시 jitter 증가
- 52. 다중 controller 동시 운용 시 priority 충돌 — 명시적 chaining 필요

6. 대안 비교

항목	ros2_control	ROS 1 ros_control	Custom CAN driver	Roboticslab
ROS 2 native	✓	✗	—	별도
MovelT 2 통합	✓	✗EOL	수동	별도
1 kHz	✓PREEMPT_RT	✓	✓	—
OpenArm 공식	✓Enactic	✗	✗	✗
FALCON-1	✓MUST	비권장	비권장	비권장

7. FALCON-1 적용 결정

결정 항목	값
채택 여부	MUST 채택
도입 시점	Sprint 2 (HW Setup)
L1-Right	controller_manager + OpenArm Right hardware_interface
L1-Left	controller_manager + OpenArm Left hardware_interface
Update rate	1000 Hz (PREEMPT_RT)
Controller	joint_trajectory + joint_state_broadcaster + gripper_action
Hardware interface	Enactic openarm_hardware (USB-CAN-FD)
주요 의존성	OpenArm USB-CAN 어댑터, B2 PTP (양팔 timestamp 동기), A1 ROS 2

결정 항목	값
Sprint Backlog 영향	Sprint 2-3에 ~12 stories

8. 참고

- 공식 문서 : <https://control.ros.org/jazzy/index.html>
- GitHub : https://github.com/ros-controls/ros2_control
- Tutorial : https://control.ros.org/jazzy/doc/ros2_control_demos/doc/index.html
- OpenArm hardware_interface : https://github.com/enactic/openarm_ros2 (가정)
- PREEMPT_RT 커널 빌드 : <https://wiki.linuxfoundation.org/realtime/start>

D6. MoveIt 2 Servo

BLUF ★★★★★ SHOULD (Sprint 4 검토). Cartesian velocity jog 실시간 제어, MoveIt 2 plan-and-execute의 reactive 보완. v1.0 Pick-and-Place는 plan-and-execute로 충분, 그러나 fine-positioning (handover 마지막 5cm) 또는 사람 손 추적 (MediaPipe Hands 결합) 시 Servo 가치. RPi5에서 50-100Hz 동작.

1. 개요

항목	내용
풀네임	MoveIt 2 Servo
라이선스	BSD
관리	PickNik Robotics
역할	Cartesian velocity → joint velocity 실시간 변환
입력	TwistStamped (Cartesian velocity command)

항목	내용
출력	joint_trajectory (1 sample) → ros2_control
주기	100-500 Hz

2. 적용 방법

```
sudo apt install ros-jazzy-moveit-servo

# launch
ros2 launch moveit_servo demo.launch.py

# Twist 명령 발행 (joystick, gamepad, 또는 perception)
ros2 topic pub /servo_node/delta_twist_cmds geometry_msgs/msg/TwistStamped \
    "{header: {frame_id: 'base_link'}, twist: {linear: {x: 0.05}}}"
```

3. FALCON-1 사용 시나리오

- Handover Option H3 (영역 G-H 결합) — wrist D405 가 사람 손 추적, MediaPipe Hands → Twist 명령으로 ext-end
- Fine-positioning — Pick-and-Place 마지막 5cm 미세 조정
- Teleop demonstration 수집 (v1.1 imitation learning) — 작업자가 joystick 으로 OpenArm 조작, 50 회 데이터 수집
- Compliance behavior — 사람이 OpenArm 을 직접 밀면 backdrivable + Servo 로 부드럽게 반응 (협업 narrative 강화)

4. 하드웨어 필요·제약

항목	요구	FALCON-1 적합성
CPU	duo+ 2 GHz	✔RPI5 또는 L1
RAM	200 MB	✔
주기	100-500 Hz	✔RPI5에서 100 Hz 가능

항목	요구	FALCON-1 적합성
IK 솔버	TRAC-IK 권장	□ KDL도 가능
ros2_control	필수	✓ D5

5. 장점·단점 요약

장점 :

- Cartesian 직접 제어 — 사용자가 직관적으로 "앞으로 5cm" 명령
- 실시간 reactive — perception 기반 추적에 적합
- Singularity-collision avoidance 내장 — velocity scaling 으로 안전
- Joystick teleop 표준 — Mobile ALOHA 같은 데이터 수집 패턴

단점 :

- Plan-and-execute 보다 결정성 약함 — 미리 trajectory 검증 안 됨
- Whole-task 시나리오 부적합 — Pick-and-Place 는 MoveIt 2 + MTC 가 정답
- Tuning 까다로움 — gain · scale · timeout 등
- 디버깅 어려움 — 실시간 변환이라 trace 부족

6. FALCON-1 적용 결정

결정 항목	값
채택 여부	SHOULD — Sprint 4 검토
도입 시점	Sprint 4 (Integrated Demo v0)
사용 시나리오	Handover fine-positioning, teleop 데이터 수집 (v1.1)
처리 위치	L1-Right (handover 시) 또는 RPi5
주기	100 Hz (RPi5) 또는 200 Hz (L1)

결정 항목	값
주요 의존성	D1 MoveIt 2, D5 ros2_control

7. 참고

- 공식 : https://moveit.picknik.ai/main/doc/examples/realtime_servo/realtime_servo_tutorial.html
- GitHub : https://github.com/moveit/moveit2/tree/main/moveit_ros/moveit_servo

D7. CHOMP / STOMP

BLUF ★★ 비권장. 최적화 기반 motion planner, OMPL의 sampling 한계를 보완하나 trajectory 평활성·tunable 어려움. v1.0 PoC에는 OMPL + Pilz LIN 조합으로 충분, CHOMP/STOMP는 v1.2+ trajectory 품질 정밀 튜닝 시점에 검토.

1. 개요

항목	내용
CHOMP	Covariant Hamiltonian Optimization for Motion Planning
STOMP	Stochastic Trajectory Optimization for Motion Planning
라이선스	BSD
관리	MoveIt 2 plug-in 형태
방식	Trajectory를 cost function으로 최적화
대안 위치	OMPL의 trajectory 후처리 또는 단독 planner

2. 비권장 사유

- 53. OMPL + Pilz LIN 으로 v1.0 PoC 충분 — 추가 학습·tuning 부담
- 54. Tuning 매우 까다로움 — cost weights · learning rate 등 다수 파라미터
- 55. Convergence 변동 — 일부 query 에서 local minima 에 갇힘
- 56. Trajectory 부드러움이 Pilz 대비 부족
- 57. Real-time 부적합 — 최적화 시간이 OMPL 대비 길거나 비슷
- 58. MoveIt 2 main developer (PickNik)도 OMPL + Pilz 우선

3. 검토 가치 (v1.2+)

- 동적 환경에서 obstacle 회피 trajectory 부드러움 강화
- Bimanual coordinated trajectory 품질 (양팔 trajectory blending)
- 학회 paper 에서 baseline 비교 (OMPL vs CHOMP vs STOMP)

4. FALCON-1 적용 결정

결정 항목	값
v1.0 채택	비채택 — OMPL + Pilz로 충분
v1.2+ 검토	trajectory 품질 정밀 튜닝 단계에서
참고 가치	학회 paper baseline 비교용

5. 참고

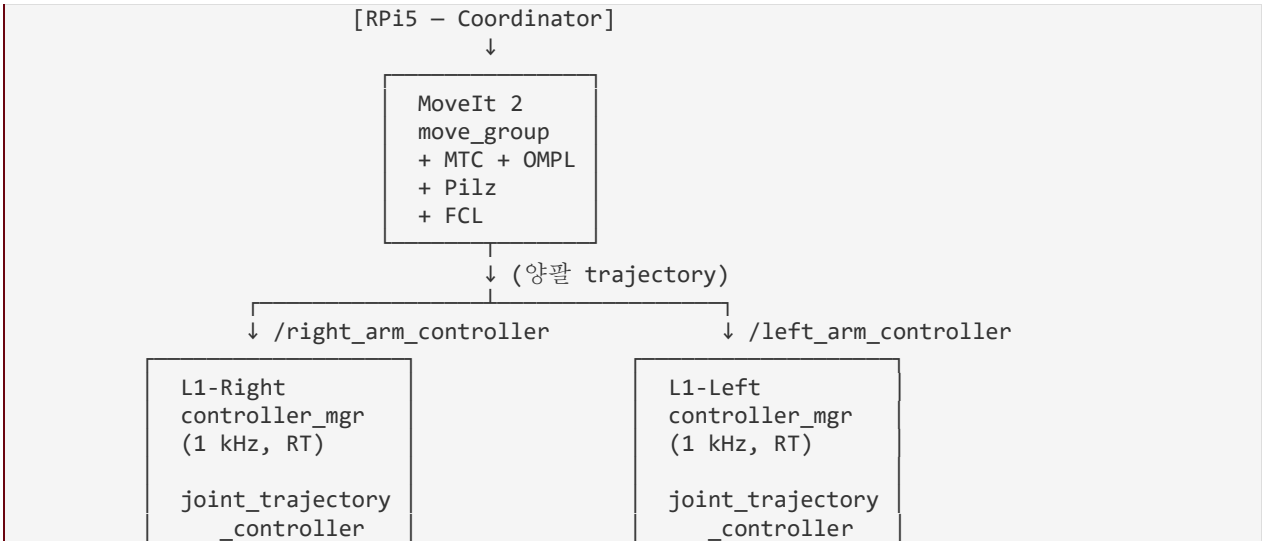
- CHOMP 논문 : Ratliff et al. 2009 ICRA
- STOMP 논문 : Kalakrishnan et al. 2011 ICRA
- MoveIt CHOMP :
https://moveit.picknik.ai/main/doc/examples/chomp_planner/chomp_planner_tutorial.html

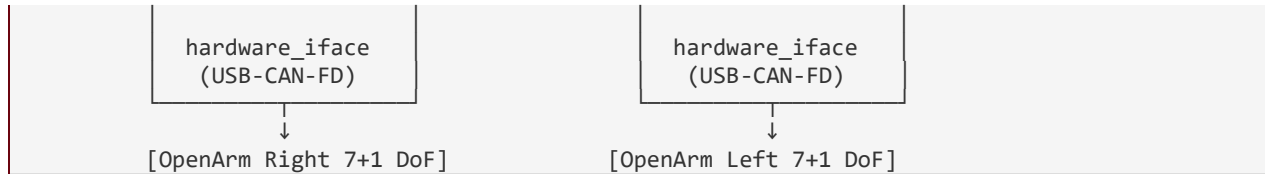
영역 D 종합 결론

1. 채택 / 비채택 정리

#	기술	v1.0 채택	도입 Sprint	역할
D1	MoveIt 2	✔MUST	Sprint 2	Manipulation framework
D2	Pilz Industrial Motion	✔MUST	Sprint 3	LIN/CIRC 산업 trajectory
D3	moveit_task_constructor	✔MUST	Sprint 3	Pick-and-Place task graph
D4	OMPL	✔MUST (자동)	Sprint 2	Sampling-based planner
D5	ros2_control	✔MUST	Sprint 2	양팔 1kHz hardware abstraction
D6	MoveIt 2 Servo	SHOULD (Sprint 4)	Sprint 4	Cartesian velocity jog
D7	CHOMP / STOMP	비채택	v1.2+	Trajectory 최적화

2. 통합 Manipulation 토폴로지 (Dual-L1 분산)





3. Pick-and-Place task graph (FALCON-1 v1.0 표준)

```

Task: dual_arm_pick_place
├─ Parallel container
│   ├── right_arm_pick (driver)
│   │   ├── Stage 1: CurrentState
│   │   ├── Stage 2: Open right gripper
│   │   ├── Stage 3: Approach (OMPL RRTConnect)
│   │   ├── Stage 4: GenerateGraspPose (AprilTag)
│   │   ├── Stage 5: Pick
│   │   │   ├── Approach descent (Pilz LIN, 10cm)
│   │   │   ├── Close gripper
│   │   │   └── Lift up (Pilz LIN, 5cm)
│   │   ├── Stage 6: Transport to Box-D (OMPL)
│   │   ├── Stage 7: Place
│   │   │   ├── Descent (Pilz LIN, 10cm)
│   │   │   ├── Open gripper
│   │   │   └── Retreat (Pilz LIN, 5cm)
│   │   └── Stage 8: Return home (OMPL)
│   └── left_arm_pick (hammer) [동시 실행, 충돌 회피]
│       └── (동일 stage 구조)
  
```

4. Sprint 2-4 진입 작업 list

59. Sprint 2 : MoveIt 2 + ros2_control 설치 (L1×2 + RPi5)
60. Sprint 2 : OpenArm 공식 URDF + SRDF 양팔 통합 (Setup Assistant)
61. Sprint 2 : ros2_control hardware_interface 검증 — 1kHz update rate, jitter 측정
62. Sprint 2 : PREEMPT_RT 커널 빌드 + 적용 (L1×2)
63. Sprint 2 : 단일 팔 PTP motion 검증 (joint_trajectory_controller)
64. Sprint 3 : Pilz LIN motion 검증 — 직선 trajectory 정확도
65. Sprint 3 : moveit_task_constructor pick subtask 작성 + 시험
66. Sprint 3 : 양팔 동시 plan + collision avoidance 검증
67. Sprint 4 (선택) : MoveIt Servo 통합 — handover fine-positioning
68. Sprint 5 burn-in : 양팔 동시 pick-place 100 회 반복, 성공률 측정

5. 잔여 위험 및 대응

#	위험	심각도	대응
1	PREEMPT_RT 커널 + NVIDIA driver 호환성	High	Sprint 2 안에 검증, 실패 시 표준 커널 + RT thread
2	RPi5 MoveIt + Nav2 동시 운용 부하	Med-High	Sprint 3 측정 후 Coordinator를 L1-Right로 이전 검토
3	양팔 동시 trajectory 충돌 회피 실패	Med	MTC Alternatives container로 retry, 양팔 timing 분리 fallback
4	OpenArm hardware_interface 미공개 시	Med	Enactic 문의 또는 자체 작성 (~30h 추가)
5	Singularity 시 trajectory 실패	Low	TRAC-IK 도입 (Sprint 4) 또는 joint limit 보수적 설정
6	AprilTag 기반 grasp pose 정확도 (영역 H)	Low	Wrist D405 close-range fine-tuning (Hybrid Hand-coded grasp)

6. 검증 시험 (Test Plan 매핑)

Test ID	내용	Sprint	성공 기준
T-MAN-01	ros2_control 1kHz jitter	Sprint 2	jitter < 100 μ s (95%)
T-MAN-02	단일 팔 PTP 정확도	Sprint 2	TCP 위치 $\pm$ 2 mm
T-MAN-03	Pilz LIN trajectory 정확도	Sprint 3	직선 deviation < 1 mm
T-MAN-04	양팔 동시 collision avoidance	Sprint 3	100회 시도, 충돌 0회
T-MAN-05	MTC Pick-and-Place 성공률	Sprint 4	$\geq$ 90% (retry 1회 시 $\geq$ 95%)
T-MAN-06	Servo 추적 latency (선택)	Sprint 4	< 50 ms

Test ID	내용	Sprint	성공 기준
T-MAN-07	Burn-in 100회 반복	Sprint 5	성공률 $\geq 90\%$, 평균 cycle time < 30s

7. 다음 작성 영역

영역 D Manipulation 제어 7종 분석 완료. 후속 영역 :

- 69. 영역 E — Perception Vision (12 종) : YOLOv11/v12, RT-DETR, MediaPipe Hands/Pose, AprilTag, ArUco, SAM 2, Grounding DINO, OpenPose, DeepSORT 등
- 70. 영역 F — Perception Depth · 3D (6 종)
- 71. 영역 G — HRI · 음성 (8 종)
- 72. 영역 H — Grasp Planning (6 종)
- 73. 영역 I — LLM · Reasoning (6 종)
- 74. 영역 J — Learning RL · Imitation · VLA (8 종)
- 75. 영역 K — Sim · CI · Logging (8 종)
- 76. 영역 L — Safety · Power (6 종)

부록 — 변경 이력

버전	일자	내용	작성
v0.1	2026-04-30	영역 D Manipulation 제어 7종 초안 작성 (BURGUNDY accent)	이건희, CTO 페르소나 검토